

# Rechnernetze – Übung 6

## Majd Abbas

### Aufgabe 1a: Sliding-Window und TCP-Varianten

#### Sliding-Window

Sliding-Window ist ein Verfahren zur Flusskontrolle und zur effizienteren Datenübertragung. Die Grundidee ist, dass der Sender nicht nach jedem einzelnen Paket warten muss, bis eine Bestätigung eintrifft. Stattdessen darf er mehrere Pakete hintereinander senden, solange sie innerhalb eines bestimmten Fensters liegen.

Dieses Fenster beschreibt die Menge der Pakete, die bereits gesendet, aber noch nicht bestätigt wurden. Sobald eine Bestätigung für ein Paket ankommt, verschiebt sich das Fenster weiter nach rechts. Dadurch kann der Sender kontinuierlich Daten übertragen, ohne den Empfänger unbegrenzt zu überlasten. Man braucht Sliding-Window also, um die Leitung besser auszunutzen und trotzdem zu verhindern, dass zu viele unbestätigte Pakete gleichzeitig unterwegs sind.

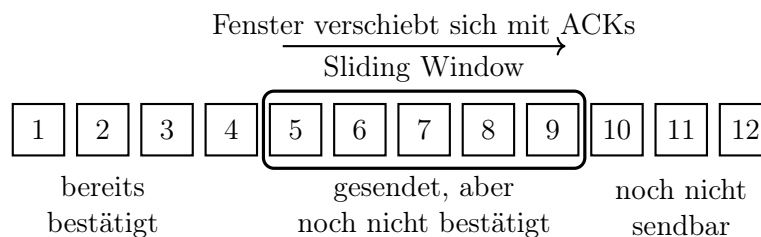


Abbildung 1: Beispiel für ein Sliding-Window: Der Sender darf nur Pakete innerhalb des Fensters unbestätigt offen haben.

#### TCP Tahoe

TCP Tahoe ist eine frühe TCP-Variante zur Überlastkontrolle. TCP Tahoe verwendet vor allem *Slow Start*, *Congestion Avoidance* und *Fast Retransmit*. Am Anfang wächst das Congestion Window schnell, damit die verfügbare Bandbreite gefunden wird. Sobald ein Verlust erkannt wird, zum Beispiel durch Timeout oder mehrfach gleiche ACKs, interpretiert Tahoe das als Zeichen für Überlastung.

Danach setzt Tahoe das Congestion Window wieder stark zurück, meistens auf ein Segment, und beginnt erneut mit Slow Start. Tahoe ist dadurch vorsichtig, kann aber nach einem Verlust relativ viel Übertragungsrate verlieren.

#### TCP Reno

TCP Reno erweitert Tahoe um *Fast Recovery*. Wenn Reno drei doppelte ACKs erhält, geht es davon aus, dass ein einzelnes Segment verloren gegangen ist, aber die Verbindung nicht komplett zusammengebrochen ist. Deshalb wird das Congestion Window nicht ganz auf den Anfang zurückgesetzt, sondern ungefähr halbiert.

Bei einem Timeout reagiert Reno aber weiterhin streng und beginnt wieder mit Slow Start. Reno ist also effizienter als Tahoe, weil es sich nach einzelnen Paketverlusten schneller erholt.

## TCP Vegas

TCP Vegas verfolgt einen anderen Ansatz als Tahoe und Reno. Tahoe und Reno reagieren vor allem auf Paketverluste. Vegas versucht dagegen schon vorher zu erkennen, ob eine Überlastung entsteht. Dafür vergleicht Vegas die erwartete Datenrate mit der tatsächlich gemessenen Datenrate. Wenn die tatsächliche Rate deutlich schlechter wird, deutet das auf wachsende Warteschlangen im Netz hin.

Dann reduziert Vegas das Congestion Window früher, also bevor viele Pakete verloren gehen. TCP Vegas ist damit stärker verzögerungsbasiert, während Tahoe und Reno eher verlustbasiert arbeiten.

## Aufgabe 1b: Protokolle und ISO/OSI-Schichten

| Protokoll            | OSI-Schicht | Kurze Begründung                                                                                                                                  |
|----------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Ethernet             | Schicht 1/2 | Ethernet beschreibt sowohl die physikalische Übertragung über Medien als auch Frames und MAC-Adressen auf der Sicherungsschicht.                  |
| IEEE 802 / MAC / LLC | Schicht 2   | Diese Konzepte gehören zur Sicherungsschicht, weil sie den Zugriff auf das Medium und die lokale Übertragung von Frames regeln.                   |
| ARP                  | Schicht 2/3 | ARP verbindet IP-Adressen mit MAC-Adressen. Es arbeitet also zwischen Netzwerkschicht und Sicherungsschicht.                                      |
| RARP                 | Schicht 2/3 | RARP ist das Gegenstück zu ARP und ordnet einer MAC-Adresse eine IP-Adresse zu.                                                                   |
| IP / IPv4            | Schicht 3   | IP adressiert Hosts in verschiedenen Netzen und ermöglicht Routing zwischen Netzwerken.                                                           |
| IPv6                 | Schicht 3   | IPv6 ist wie IPv4 ein Internetprotokoll der Netzwerkschicht, aber mit 128-Bit-Adressen.                                                           |
| ICMP                 | Schicht 3   | ICMP gehört zur Netzwerkschicht, weil es Kontroll- und Fehlermeldungen für IP überträgt, zum Beispiel bei Erreichbarkeitsproblemen.               |
| Routingprotokolle    | Schicht 3   | Routingprotokolle berechnen Wege durch ein Netzwerk und gehören deshalb zur Netzwerkschicht.                                                      |
| UDP                  | Schicht 4   | UDP nutzt IP plus Portnummern und bietet eine einfache, verbindungslose Transportmöglichkeit ohne Garantie für Reihenfolge oder Zustellung.       |
| TCP                  | Schicht 4   | TCP bietet einen zuverlässigen Bytestrom mit ACKs, Wiederholungen, Flusskontrolle und Überlastkontrolle.                                          |
| DNS                  | Schicht 7   | DNS ist ein Anwendungsprotokoll zur Übersetzung von Namen in IP-Adressen und umgekehrt.                                                           |
| DHCP                 | Schicht 7   | DHCP wird von Anwendungen bzw. Systemdiensten genutzt, um automatisch Netzwerkkonfigurationen wie IP-Adresse, Gateway und DNS-Server zu erhalten. |

| Protokoll | OSI-Schicht                             | Kurze Begründung                                                                                                                           |
|-----------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| NAT       | eher Schicht 3/4                        | NAT ist kein klassisches End-zu-End-Protokoll, sondern eine Technik, die IP-Adressen und oft auch Portnummern beim Weiterleiten verändert. |
| HTTP      | Schicht 7                               | HTTP ist ein Anwendungsprotokoll für Webseiten und Webkommunikation.                                                                       |
| TLS/SSL   | Schicht 5/6, praktisch zwischen 4 und 7 | TLS sichert Anwendungsdaten durch Verschlüsselung und Authentifizierung, liegt praktisch zwischen TCP und Anwendungsprotokollen wie HTTP.  |

## Aufgabe 2: Nmap

### 2a) Hosts im lokalen Klasse-C-Netz

Zuerst habe ich mit `ipconfig` meine lokale IPv4-Adresse und die Subnetzmaske bestimmt. Mein aktiver Netzwerkadapter war Wi-Fi. Die IPv4-Adresse war `192.168.1.94` und die Subnetzmaske war `255.255.255.0`. Daher habe ich das lokale Klasse-C-Netz als `192.168.1.0/24` betrachtet.

Anschließend habe ich folgenden Nmap-Befehl verwendet:

```
nmap -sn 192.168.1.0/24
```

Die Option `-sn` führt eine Host-Erkennung durch. Es werden also keine Ports gescannt, sondern nur erreichbare Hosts gesucht.

Nmap lieferte folgendes Ergebnis:

```
Nmap done: 256 IP addresses (3 hosts up) scanned in 5.06 seconds
```

Damit befinden sich in meinem lokalen Klasse-C-Netz `192.168.1.0/24` insgesamt 3 aktive Hosts. Gefunden wurden der Router `192.168.1.1`, mein eigener Rechner `192.168.1.94` und ein weiteres Gerät mit der Adresse `192.168.1.114`.

In Wireshark waren während des Scans vor allem ARP-Pakete zu sehen. Mein Rechner mit der Adresse `192.168.1.94` fragte per Broadcast nach den MAC-Adressen verschiedener IP-Adressen im lokalen Netz, zum Beispiel durch Nachrichten der Form `Who has 192.168.1.1?` `Tell 192.168.1.94`. Antwortet ein Gerät auf diese ARP-Anfrage, kann Nmap erkennen, dass dieser Host im lokalen Netz aktiv ist.

### 2b) Betriebssystem von `scanme.nmap.org`

Zur Betriebssystemerkennung habe ich PowerShell als Administrator geöffnet und folgenden Nmap-Befehl verwendet:

```
nmap -O -v scanme.nmap.org
```

Die Option `-O` aktiviert die Betriebssystemerkennung. Dabei sendet Nmap verschiedene Testpakete an das Zielsystem und vergleicht die Antworten mit bekannten Betriebssystem-Fingerprints. Die Option `-v` steht für *verbose* und zeigt während des Scans zusätzliche Informationen an.

Nmap lieferte für `scanme.nmap.org` die IP-Adresse `45.33.32.156` und erkannte folgendes Betriebssystem:

```
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.14
```

Damit verwendet `scanme.nmap.org` wahrscheinlich ein Linux-System mit einem Kernel aus dem Bereich 5.0 bis 5.14.

Zusätzlich wurden beim Scan mehrere offene TCP-Ports gefunden:

| Port      | Status | Dienst     |
|-----------|--------|------------|
| 22/tcp    | open   | ssh        |
| 80/tcp    | open   | http       |
| 9929/tcp  | open   | nping-echo |
| 31337/tcp | open   | Elite      |

In Wireshark habe ich den Filter

```
ip.addr == 45.33.32.156
```

verwendet. Dort waren TCP-Pakete zwischen meinem Rechner 192.168.1.94 und 45.33.32.156 zu sehen. Besonders auffällig waren Antworten wie SYN, ACK von offenen Ports, zum Beispiel von Port 22. Solche Antworten zeigen, dass der entsprechende TCP-Port erreichbar ist.

## 2c) Registrierungsdatum von nmap.org

Da Windows `whois` nicht standardmäßig installiert hatte, habe ich ein Whois-Werkzeug verwendet und anschließend folgenden Befehl ausgeführt:

```
whois nmap.org
```

Im Whois-Ergebnis stand unter anderem:

```
Domain Name: NMAP.ORG
```

```
Created Date: 1999-01-18T05:00:00Z
```

Die Domain `nmap.org` wurde also am 18.01.1999 registriert.

## 2d) Effektiver Scan vieler Adressen nach offenen TCP-Ports

Um eine größere Menge an Adressen möglichst effektiv nach offenen TCP-Ports zu scannen, habe ich nicht alle 65535 TCP-Ports geprüft, sondern den Scan auf die 100 häufigsten TCP-Ports beschränkt. Dafür habe ich folgenden Befehl verwendet:

```
nmap -T4 --top-ports 100 192.168.1.0/24
```

Die Option `-T4` erhöht die Scan-Geschwindigkeit. Das ist in einem lokalen Netz sinnvoll, weil die Latenz gering ist und Paketverluste normalerweise nicht stark auftreten. Die Option `--top-ports 100` bedeutet, dass nur die 100 häufigsten TCP-Ports gescannt werden. Dadurch ist der Scan deutlich schneller als ein vollständiger Scan aller 65535 TCP-Ports auf allen 256 Adressen des Netzes.

Der Scan lieferte folgendes Ergebnis:

```
Nmap scan report for o2.box (192.168.1.1)
```

```
Host is up (0.0036s latency).
```

```
Not shown: 95 closed tcp ports (reset)
```

```
PORT      STATE SERVICE
```

```
53/tcp    open  domain
```

```
80/tcp    open  http
```

```
139/tcp   open  netbios-ssn
```

```
445/tcp   open  microsoft-ds
```

```
49152/tcp open  unknown
```

```
MAC Address: 2C:EA:DC:E0:8E:21 (Askey Computer)
```

```
Nmap scan report for DESKTOP-4K9U8MC (192.168.1.94)
```

```
Host is up (0.000010s latency).
```

```
Not shown: 97 closed tcp ports (reset)
```

```
PORT      STATE SERVICE
```

```
135/tcp   open  msrpc
```

```
139/tcp   open  netbios-ssn
```

```
445/tcp   open  microsoft-ds
```

```
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.22 seconds
```

Damit ist diese Methode effizient, weil viele Adressen in kurzer Zeit geprüft werden können, ohne jeden möglichen Port einzeln zu scannen. In Wireshark waren dabei viele TCP-Pakete zu sehen. Typisch sind SYN-Pakete an verschiedene Zielpports. Antwortet ein Ziel mit **SYN/ACK**, ist der Port wahrscheinlich offen. Antwortet es mit **RST**, ist der Port geschlossen.

## Aufgabe 3: DHCP

### a) Verwendung von DHCP

DHCP steht für *Dynamic Host Configuration Protocol*. Das Protokoll wird verwendet, um Rechner in einem Netzwerk automatisch mit den nötigen Netzwerkinformationen zu konfigurieren. Dazu gehören zum Beispiel die eigene IP-Adresse, die Subnetzmaske, der Default-Router und der DNS-Server.

Ein anschauliches Beispiel ist ein Laptop, der sich mit einem WLAN verbindet. Am Anfang kennt der Laptop noch keine passende IPv4-Adresse für dieses Netzwerk. Deshalb fragt er im lokalen Netz nach einer Konfiguration. Der DHCP-Server, zum Beispiel der Router, weist dem Laptop dann automatisch eine Adresse und weitere Konfigurationsdaten zu. Dadurch muss der Benutzer diese Werte nicht manuell eingeben.

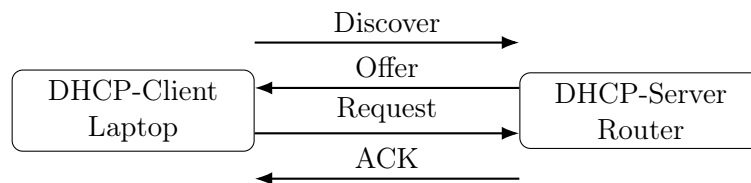


Abbildung 2: Vereinfachter Ablauf der DHCP-Kommunikation

### b) Aufzeichnung mit Wireshark

Die DHCP-Pakete wurden mit Wireshark auf der aktiven WLAN-Schnittstelle aufgezeichnet. Um neue DHCP-Pakete zu erzeugen, wurde unter Windows zuerst die aktuelle IPv4-Konfiguration freigegeben und danach erneut angefordert:

```
ipconfig /release "Wi-Fi"  
ipconfig /renew "Wi-Fi"
```

Als Capture-Filter wurde folgender Filter verwendet:

```
udp and (port 67 or port 68)
```

Dieser Filter ist geeignet, weil DHCP über UDP läuft und dabei die Ports 67 und 68 verwendet. Dadurch enthält die PCAP-Datei ausschließlich DHCP-Pakete.

In der Aufzeichnung wurden die vier wichtigen Nachrichten *DHCP Discover*, *DHCP Offer*, *DHCP Request* und *DHCP ACK* aufgezeichnet. Der DHCP-Server hatte die Adresse 192.168.1.1. Mein Rechner erhielt die IPv4-Adresse 192.168.1.94 mit der Subnetzmaske 255.255.255.0. Als Default-Gateway und DNS-Server wurde ebenfalls 192.168.1.1 angegeben.

Die Screenshots der Packet Details zu den vier DHCP-Nachrichten befinden sich in einer separaten Datei.

### c) Erklärung der DHCP-Nachrichten

#### DHCP Discover

Das DHCP-Discover-Paket ist die erste Nachricht des Clients. Der Client besitzt zu diesem Zeitpunkt noch keine gültige IPv4-Adresse und sucht deshalb nach einem DHCP-Server im lokalen Netzwerk. In der Aufzeichnung wurde dieses Paket von 0.0.0.0 an die Broadcast-Adresse 255.255.255.255 gesendet.

In den Packet Details sieht man unter anderem die Transaction-ID, die MAC-Adresse des Clients und die Option *DHCP Message Type: Discover*. Die Transaction-ID wird verwendet, um die zusammengehörigen DHCP-Nachrichten einander zuzuordnen.

## **DHCP Offer**

Mit DHCP Offer antwortet der DHCP-Server auf die Anfrage des Clients. In meiner Aufzeichnung kam diese Nachricht vom Server 192.168.1.1. Der Server bietet dem Client die Adresse 192.168.1.94 an.

Zusätzlich werden weitere Informationen übertragen, zum Beispiel die Subnetzmaske, die Lease-Time, der Router und der DNS-Server. Die Lease-Time gibt an, wie lange der Client die Adresse verwenden darf.

## **DHCP Request**

Im DHCP-Request-Paket teilt der Client mit, dass er die angebotene Konfiguration verwenden möchte. In der Aufzeichnung wurde dieses Paket wieder von 0.0.0.0 an 255.255.255.255 gesendet.

In den Packet Details sieht man die angeforderte IP-Adresse 192.168.1.94 und den ausgewählten DHCP-Server 192.168.1.1. Falls mehrere DHCP-Server geantwortet hätten, würde der Client mit dieser Nachricht zeigen, welches Angebot er annimmt.

## **DHCP ACK**

Mit DHCP ACK bestätigt der Server endgültig die Zuweisung der Adresse. Danach darf der Client die IPv4-Adresse 192.168.1.94 im Netzwerk verwenden.

Das ACK-Paket enthält erneut wichtige Konfigurationsdaten wie Subnetzmaske, Lease-Time, Router und DNS-Server. Damit ist die automatische Netzwerkkonfiguration abgeschlossen.

## Aufgabe 4: Routing

### Aufgabe 4a: Distanz-Vektor-Routing mit Bellman-Ford

Beim Distanz-Vektor-Verfahren speichert jeder Router eine Tabelle mit den aktuell bekannten Kosten zu allen Zielnetzen. Am Anfang kennt ein Router nur die Kosten zu seinen direkten Nachbarn. Danach tauschen benachbarte Router regelmäßig ihre Distanzvektoren aus.

Ein Router aktualisiert seine Tabelle mit der Bellman-Ford-Idee:

$$D_x(z) = \min_v \{c(x, v) + D_v(z)\}.$$

Dabei ist  $D_x(z)$  die aktuell beste bekannte Distanz von Router  $x$  zum Ziel  $z$ ,  $c(x, v)$  die direkte Verbindungskosten von  $x$  zum Nachbarn  $v$ , und  $D_v(z)$  die von  $v$  gemeldete Distanz zum Ziel  $z$ .

Im Beispiel aus der Aufgabenstellung gilt zum Beispiel:

$$c(A, B) = 3, \quad c(B, C) = 2, \quad c(C, D) = 5, \quad c(A, C) = 23.$$

Router  $A$  kennt zunächst nur seine direkten Wege zu  $B$  und  $C$ . Wenn  $A$  von  $B$  erfährt, dass  $B$  das Netz  $C$  mit Kosten 2 erreicht, kann  $A$  den Weg nach  $C$  über  $B$  berechnen:

$$c(A, B) + D_B(C) = 3 + 2 = 5.$$

Dieser Weg ist besser als der direkte Weg  $A \rightarrow C$  mit Kosten 23. Deshalb trägt  $A$  für das Ziel  $C$  den nächsten Hop  $B$  und die Kosten 5 ein.

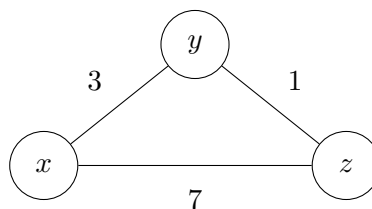
Der Unterschied zu Link-State-Verfahren besteht darin, dass beim Distanz-Vektor-Verfahren jeder Router nur mit seinen Nachbarn Tabellen austauscht. Er kennt also nicht unbedingt die gesamte Topologie. Beim Link-State-Verfahren sammeln die Router Informationen über den Zustand aller Links und berechnen daraus selbst die kürzesten Wege, zum Beispiel mit dem Dijkstra-Algorithmus. Link-State-Verfahren reagieren oft schneller und haben eine bessere Sicht auf das Netz, benötigen aber mehr Zustandsinformationen. Distanz-Vektor-Verfahren sind einfacher, können aber bei Änderungen langsamer konvergieren und Probleme wie Count-to-Infinity haben.

Eine hybride Nutzung kann sinnvoll sein, wenn verschiedene Netzbereiche unterschiedlich verwaltet werden. Innerhalb eines Netzes kann beispielsweise ein Link-State-Verfahren sinnvoll sein, während zwischen größeren Netzbereichen ein anderes Verfahren verwendet wird.

### Aufgabe 4b: Routingtabellen für $x, y, z$

Gegeben sind die Kosten:

$$c(x, y) = 3, \quad c(y, z) = 1, \quad c(x, z) = 7.$$



Nach dem ersten Austausch der Distanzvektoren ergeben sich die besten Wege:

$$x \rightarrow y = 3, \quad x \rightarrow y \rightarrow z = 3 + 1 = 4,$$

$$y \rightarrow x = 3, \quad y \rightarrow z = 1,$$

$$z \rightarrow y = 1, \quad z \rightarrow y \rightarrow x = 1 + 3 = 4.$$

Damit lauten die endgültigen Routingtabellen:

| Router | Ziel | nächster Hop / Kosten |
|--------|------|-----------------------|
| $x$    | $x$  | $- / 0$               |
| $x$    | $y$  | $y / 3$               |
| $x$    | $z$  | $y / 4$               |
| $y$    | $x$  | $x / 3$               |
| $y$    | $y$  | $- / 0$               |
| $y$    | $z$  | $z / 1$               |
| $z$    | $x$  | $y / 4$               |
| $z$    | $y$  | $y / 1$               |
| $z$    | $z$  | $- / 0$               |

Der kostengünstigste Weg von Router  $z$  zu Router  $x$  ist also:

$$z \rightarrow y \rightarrow x$$

mit Gesamtkosten

$$1 + 3 = 4.$$

#### Aufgabe 4c: Änderung der Kosten zwischen $x$ und $y$

Nun steigen die Kosten zwischen  $x$  und  $y$  von 3 auf 7. Damit gilt:

$$c(x, y) = 7, \quad c(y, z) = 1, \quad c(x, z) = 7.$$

Für den Weg von  $z$  nach  $x$  gibt es nun zwei wichtige Möglichkeiten:

$$z \rightarrow x = 7$$

und

$$z \rightarrow y \rightarrow x = 1 + 7 = 8.$$

Der direkte Weg ist jetzt günstiger.

Die neuen endgültigen Routingtabellen sind:

| Router | Ziel | nächster Hop / Kosten |
|--------|------|-----------------------|
| $x$    | $x$  | $- / 0$               |
| $x$    | $y$  | $y / 7$               |
| $x$    | $z$  | $z / 7$               |
| $y$    | $x$  | $x / 7$               |
| $y$    | $y$  | $- / 0$               |
| $y$    | $z$  | $z / 1$               |
| $z$    | $x$  | $x / 7$               |
| $z$    | $y$  | $y / 1$               |
| $z$    | $z$  | $- / 0$               |

Ja, der kostengünstigste Pfad von  $z$  nach  $x$  ändert sich. Vorher war der beste Weg

$$z \rightarrow y \rightarrow x$$

mit Kosten 4. Nach der Änderung ist der beste Weg

$$z \rightarrow x$$

mit Kosten 7.

#### Aufgabe 4d: Ausfall von Router $D$

Wenn Router  $D$  ausfällt, verlieren die direkt verbundenen Router ihre Verbindung zu  $D$ . Im unteren Graphen betrifft das vor allem die Nachbarn von  $D$ , also  $B$ ,  $C$  und  $E$ .

Bei einem Distanz-Vektor-Verfahren merken zuerst nur die direkten Nachbarn von  $D$ , dass die Verbindung nicht mehr funktioniert. Diese Router setzen ihre Distanz zu  $D$  auf unendlich beziehungsweise unerreichbar und geben diese Änderung an ihre Nachbarn weiter. Andere Router, zum Beispiel  $A$ , erfahren den Ausfall erst indirekt durch die nächsten Tabellenaustausche. Dabei kann es passieren, dass Router vorübergehend falsche Wege zu  $D$  annehmen, weil sie alte Informationen von Nachbarn erhalten. Deshalb kann die Konvergenz bei Distanz-Vektor-Verfahren langsam sein.

Bei einem Link-State-Verfahren erkennen die direkten Nachbarn von  $D$  ebenfalls zuerst den Ausfall ihrer Links. Danach verbreiten sie diese Link-State-Information im Netz. Sobald die anderen Router diese Information erhalten haben, berechnen sie ihre Wege neu. Dadurch merken normalerweise alle Router schneller und eindeutiger, dass  $D$  nicht mehr erreichbar ist.

Zusammengefasst: Beim Distanz-Vektor-Verfahren verbreitet sich die Information über den Ausfall schrittweise über Routingtabellen. Beim Link-State-Verfahren wird die Änderung des Links im Netz verteilt, und jeder Router berechnet danach seine Routingtabelle neu.